2º curso / 2º cuatr.

Grados en
Ing. Informática

Arquitectura de Computadores
Tema 1

Arquitecturas Paralelas: Clasificación y Prestaciones

Material elaborado por Mancia Anguita
Profesores: Mancia Anguita, Maribel García y Christian Morillas

UNIVERSIDAD DE GRANADA

ICAR
INGENIERÍA DE COMPUTADORES, AUTOMÁTICA Y ROBÓTICA

Lecciones

- Lección 1. Clasificación del paralelismo implícito en una aplicación
- Lección 2. Clasificación de arquitecturas paralelas
- Lección 3. Evaluación de prestaciones de una arquitectura
  - Medidas usuales para evaluar prestaciones
  - Ganancia en prestaciones
  - Eficiencia (coste, potencia, energía)
  - Conjunto de programas de prueba (*Benchmark*)

Objetivos Lección 3

- Distinguir entre tiempo de CPU (sistema y usuario) de unix y tiempo de respuesta
- Distinguir entre productividad y tiempo de respuesta
- Obtener, de forma aproximada mediante cálculos, el tiempo de CPU, GFLOPS y los MIPS del código ejecutado en un núcleo de procesamiento
- Calcular ganancia en prestaciones/velocidad
- Aplicar la ley de Amdahl
- Obtener la ganancia que supone aplicar segmentación, operación superescalar y unidades SIMD en un núcleo
- Obtener la penalización que supone los riesgos en núcleos segmentados y superescalares y cómo se eliminan por parte del compilador.
- Distinguir entre potencia y energía.
- Distinguir entre potencia máxima y TDP

Bibliografía

- Fundamental
  - Capítulo 1, M. Anguita, J. Ortega. Fundamentos y problemas de Arquitectura de Computadores, Editorial Técnica Avicam. ESIIT/C.1 ANG fun
  - Secciones 1.2,1.4, 7.5.1. J. Ortega, M. Anguita, A. Prieto. *Arquitectura de Computadores*, Thomson, 2005. ESIIT/C.1 ORT arq
  - Sección 1.6, pág. 28 John L. Hennessy, David A. Patterson, Christos Kozyrakis, *Computer Architecture : a Quantitative Approach*, Seventh edition., Waltham : Elsevier 2026.
- Complementaria
  - T. Rauber, G. Ründer. *Parallel Programming: for Multicore and Cluster Systems*. Springer 2010. Disponible en línea (biblioteca UGR): http://dx.doi.org/10.1007/978-3-642-04818-0

Contenido

- Lección 1. Clasificación del paralelismo implícito en una aplicación
- Lección 2. Clasificación de arquitecturas paralelas
- Lección 3. Evaluación de prestaciones y eficiencia de una arquitectura
  - **Medidas usuales para evaluar prestaciones**
    - Tiempo de respuesta
    - Productividad
  - Ganancia en prestaciones al realizar una mejora
  - Eficiencia (coste, potencia, energía)
  - Conjunto de programas de prueba (*Benchmark*)

Tiempo de respuesta de un programa en una arquitectura

- Real (*wall-clock time, elapsed time, real time*)
- *CPU time* = *user* + *sys* (no incluye todo el tiempo)
- Con un flujo de instrucciones
  - elapsed >= CPU time
- Con múltiples flujos de instrucciones
  - elapsed < CPU time, elapsed >= CPU time/nº_flujos_instrucciones

```
$ time ./program.exe
elapsed 5.4
user 3.2
sys 1.0
```

Elapsed >= CPU time

Tiempo de CPU de usuario (Tiempo en ejecución en espacio de usuario)

Tiempo de CPU de sistema (Tiempo en el nivel del kernel del SO)

Tiempo asociado a las esperas debidas a I/O o asociados a la ejecución de otros programas.

Comando **time** en Unix: 3.2u 1.0s 5.4

3.2+1.0 es el 78% del tiempo transcurrido (5.4)

Nota Histórica

Tiempo de CPU

```
...
for (i=0; i<N; i++) {
   v3[i]=v1[i]+v2[i];
}
...
```

$$T_{CPU} = NI \times CPI \times T_{ciclo}$$

$$T_{CPU} = NI \times \frac{1}{IPC} \times T_{ciclo}$$

$$T_{CPU} = \frac{\text{Nº}ciclos}{código} \times T_{ciclo}$$

$$T_{CPU} = \left[\sum_i NI_i \times CPI_i\right] \times T_{ciclo}$$

$$T_{CPU} = NI \times \left(\frac{\sum_i NI_i \times CPI_i}{NI}\right) \times T_{ciclo}$$

$$T_{CPU} = NI \times \quad CPI \quad \times T_{ciclo}$$

Suma vectores de *doubles*

```
...
.L7:                    ; rax=0, rbx=8N
        movsd   v1(%rax), %xmm0
        addsd   v2(%rax), %xmm0
        movsd   %xmm0, v3(%rax)
        addq    $8, %rax
        cmpq    %rbx, %rax
        jne     .L7
...
```

i	NI_i	CPI_i
movsd m,r movsd r,m	2N	4
addsd m,r	N	5
addq i,r	N	1
cmp r,r	N	1
jne	N	1
	6N	

$$T_{ciclo} = 1/F$$

Para $N = 10^3$ y $F = 100MHz$ ($\Rightarrow T_{ciclo} = 10^{-8}$seg./ciclo):

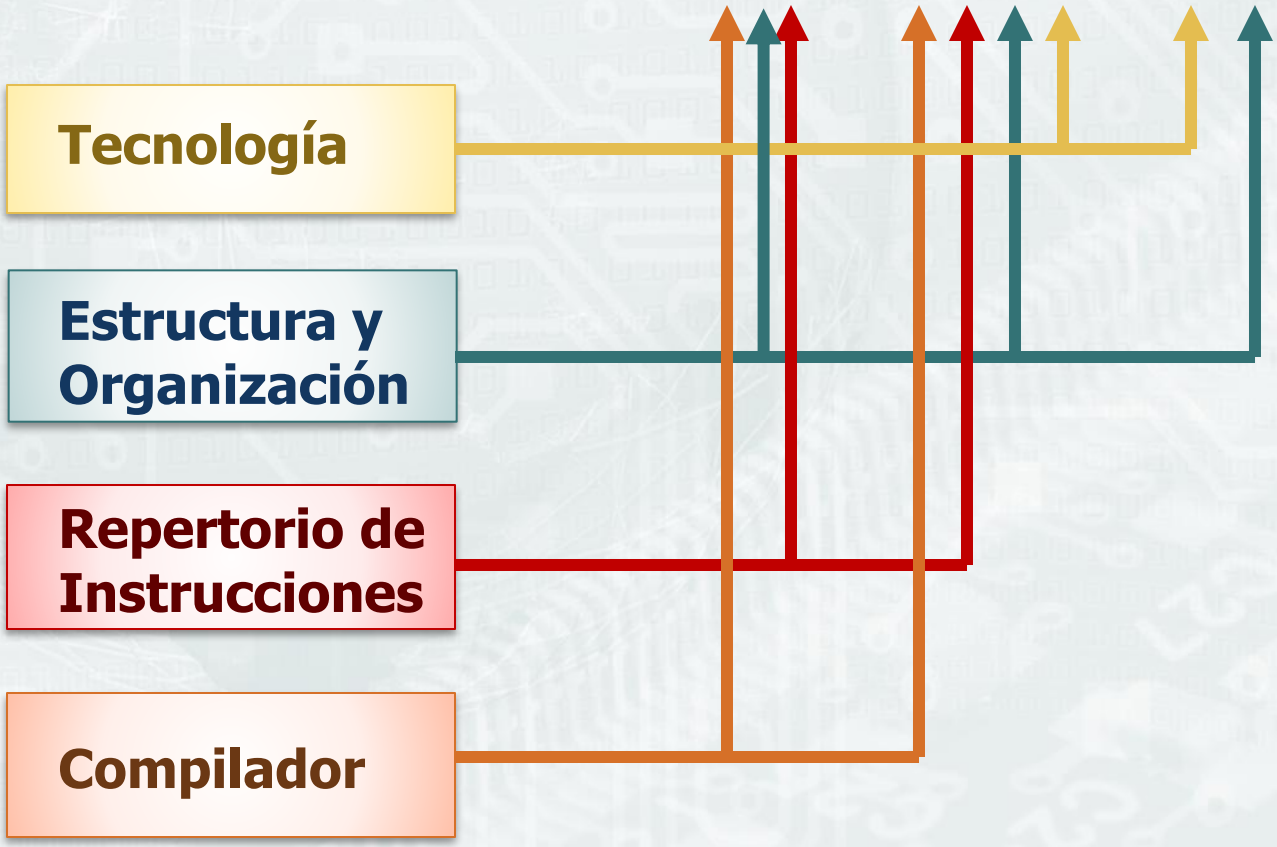
$$T_{CPU} \approx \left[6N \times \left(\frac{2N \times 4 + N \times 5 + 3N \times 1}{6N}\right)\right] \times T_{ciclo}$$

$$= 10^3 \times 16\ ciclos/código \times 10^{-8} seg./ciclo$$

$$= 16 \times 10^{-5} seg./\text{código}$$

Tiempo de CPU

$$T_{CPU} = NI \times CPI \times T_{ciclo}$$



MIPS y MFLOPS

- Millones de instrucciones por segundo (MIPS):

$$MIPS = \frac{NI}{T_{CPU} \times 10^6}$$

$$MIPS = \frac{NI}{NI \times CPI \times T_{ciclo} \times 10^6} = \frac{F}{CPI \times 10^6}$$

  - **Depende del repertorio de instrucciones** (difícil la comparación de máquinas con repertorios distintos)
  - **Puede variar inversamente con las prestaciones** (mayor valor de MIPS corresponde a peores prestaciones)

- Millones de operaciones punto flotante por segundo (MFLOPS):

$$MFLOPS = \frac{n^{\underline{o}}\ FP}{T_{CPU} \times 10^6}$$

MIPS y FLOPS

```
...
for (i=0; i<N; i++) {
   y[i]=a*x[i]+y[i];
}
...
```

-O2

```
;r12=&x,r13=&y,rax=0,rbp=N,xmm1=a
.L6:
  movsd (%r12,%rax,8), %xmm0
  mulsd %xmm1, %xmm0
  addsd (%r13,%rax,8), %xmm0
  movsd %xmm0, (%r13,%rax,8)
  addq  $1, %rax
  cmpl  %eax, %ebp
  jg .L6
```

$T(N=2^{26})=0.182$ seg.

$$GIPS = \frac{NI}{T_{CPU} \times 10^9} = \frac{N \times 7}{0.182 \times 10^9} = \frac{2^{26} \times 7}{0.182 \times 10^9} \approx 2.58\ GIPS$$

$$GFLOPS = \frac{nº\ FP}{T_{CPU} \times 10^9} = \frac{N \times 2}{0.182 \times 10^9} = \frac{2^{26} \times 2}{0.182 \times 10^9} \approx 0.737\ GFLOPS$$

-O3

```
;r12=&x,r13=&y,rax=0,rbp=N/2,xmm1=a
.L7:
  movapd (%r12), %xmm0
  addq   $1, %rax
  addq   $16, %r12
  addq   $16, %r13
  mulpd  %xmm1, %xmm0
  addpd  -16(%r13), %xmm0
  movaps %xmm0, -16(%r13)
  cmpl   %ebp, %eax
  jb .L7
```

$T(N=2^{26})=0.178$ seg.

$$GIPS = \frac{NI}{T_{CPU} \times 10^9} = \frac{(N/2) \times 9}{0.178 \times 10^9} = \frac{2^{25} \times 9}{0.178 \times 10^9} \approx 1.7 GIPS$$

$$GFLOPS = \frac{2^{26} \times 2}{0.178 \times 10^9} \approx 0.754\ GFLOPS$$

Contenido

- Lección 1. Clasificación del paralelismo implícito en una aplicación
- Lección 2. Clasificación de arquitecturas paralelas
- Lección 3. Evaluación de prestaciones y eficiencia de una arquitectura
  - Medidas usuales para evaluar prestaciones
  - **Ganancia en prestaciones al realizar una mejora**
  - Eficiencia (coste, potencia, energía)
  - Conjunto de programas de prueba (*Benchmark*)

Mejora o Ganancia en Prestaciones (*Speed-up* o ganancia en velocidad)

Si se incrementan las prestaciones de un sistema, el incremento en prestaciones (velocidad) que se consigue en la nueva situación, ***p***, con respecto a la previa (**sistema base**, ***b***) se expresa mediante la ganancia en prestaciones o *speed-up*, ***S***

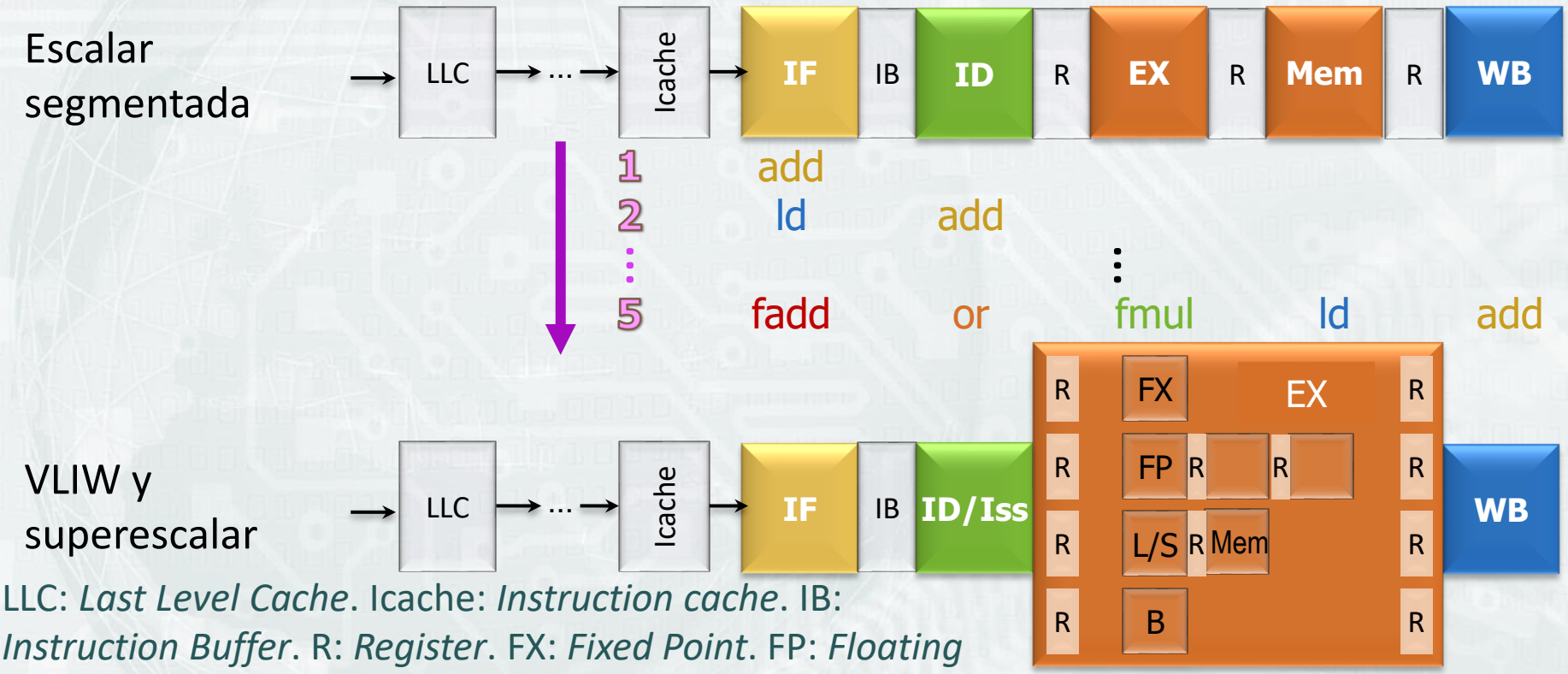
$$S = \frac{V_p}{V_b} = \frac{T_b}{T_p}$$

$$S = \frac{T_{CPU}^{b}}{T_{CPU}^{p}} = \frac{NI^b \times CPI^b \times T_{ciclo}^{b}}{NI^p \times CPI^p \times T_{ciclo}^{p}}$$

V_b Velocidad de la máquina base
V_p Velocidad de la máquina mejorada (un factor p en uno de sus componentes)
T_b Tiempo de ejecución en la máquina base
T_p Tiempo de ejecución en la máquina mejorada

Arquitecturas con paralelismo a nivel de instrucción (ILP)

Nota Histórica



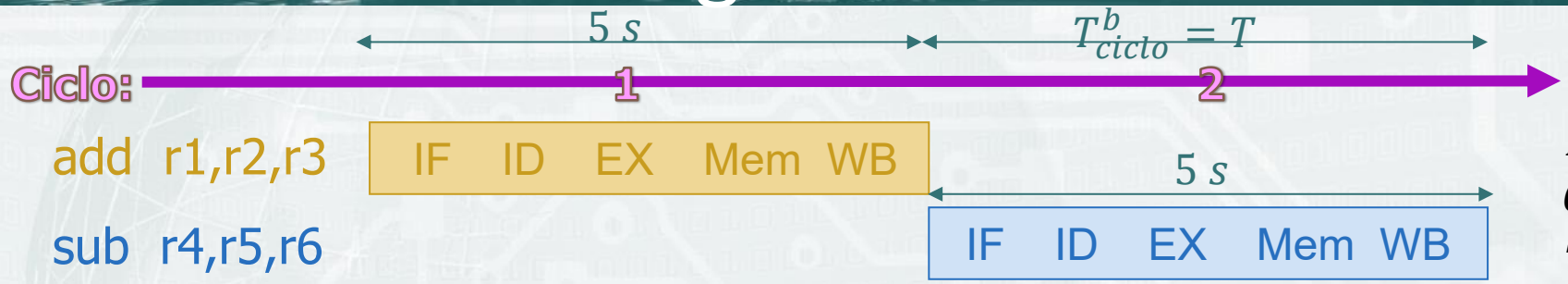
LLC: *Last Level Cache*. Icache: *Instruction cache*. IB: *Instruction Buffer*. R: *Register*. FX: *Fixed Point*. FP: *Floating Point*. L/S: (*memory) Load/Store*. B: *Branch*

https://webriscv.dii.unisi.it/

- Etapa de captación de instrucciones (***Instruction Fetch***)
- Etapa de decodificación de instrucciones y emisión a unidades funcionales (***Instruction Decode/Instruction Issue)*** (incluye captura de "operandos" de los registros de la arquitectura)
- Etapas de ejecución (***Execution***). Etapa de acceso a memoria (***Memory***)
- Etapa de almacenamiento de resultados en registros de la arquitectura (***Write-Back***)

Mejora en un núcleo de procesamiento: segmentación

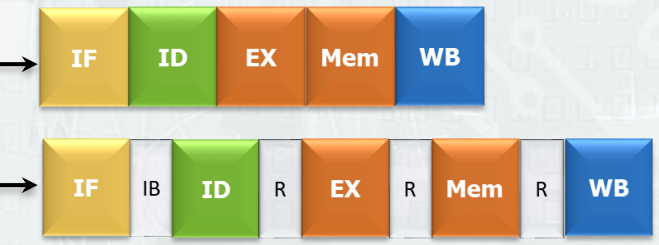
Nota Histórica



$NI^b = NI$

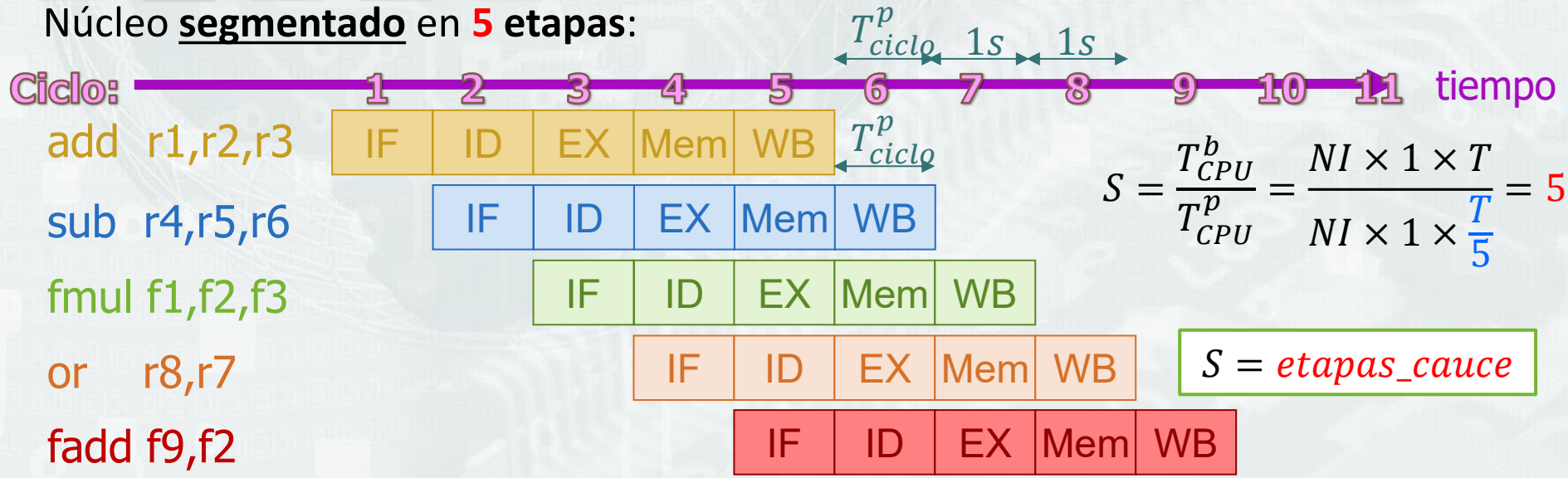
$CPI^b = 1$

$T^b_{ciclo} = T$



$$S = \frac{T^b_{CPU}}{T^p_{CPU}} = \frac{NI^b \times CPI^b \times T^b_{ciclo}}{NI^p \times CPI^p \times T^p_{ciclo}}$$

Núcleo **segmentado** en **5 etapas**:

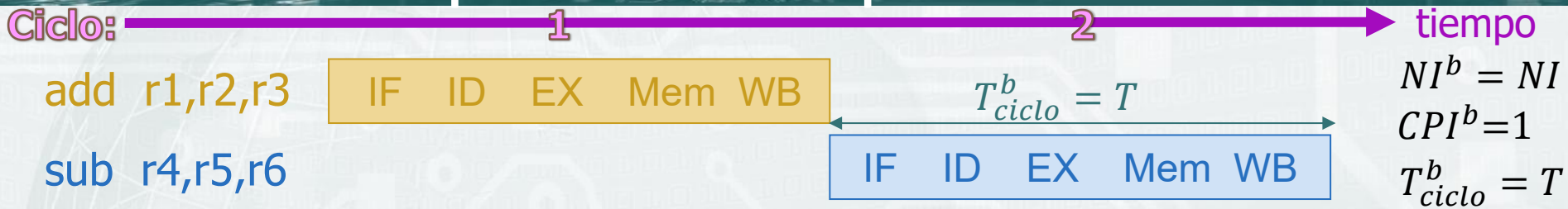


$$S = \frac{T^b_{CPU}}{T^p_{CPU}} = \frac{NI \times 1 \times T}{NI \times 1 \times \frac{T}{5}} = 5$$

$$S = etapas_cauce$$

Mejora en un núcleo de procesamiento: operación superscalar

Nota Histórica



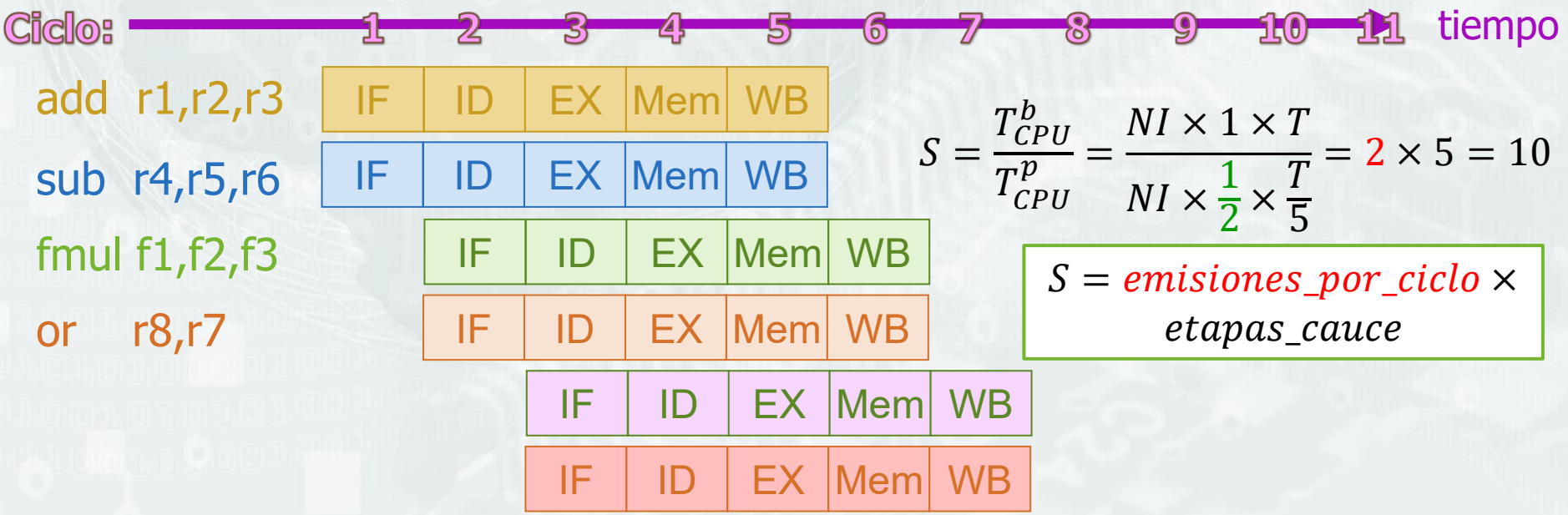
$NI^b = NI$

$CPI^b = 1$

$T^b_{ciclo} = T$

$$S = \frac{T^b_{CPU}}{T^p_{CPU}} = \frac{NI^b \times CPI^b \times T^b_{ciclo}}{NI^p \times CPI^p \times T^p_{ciclo}}$$

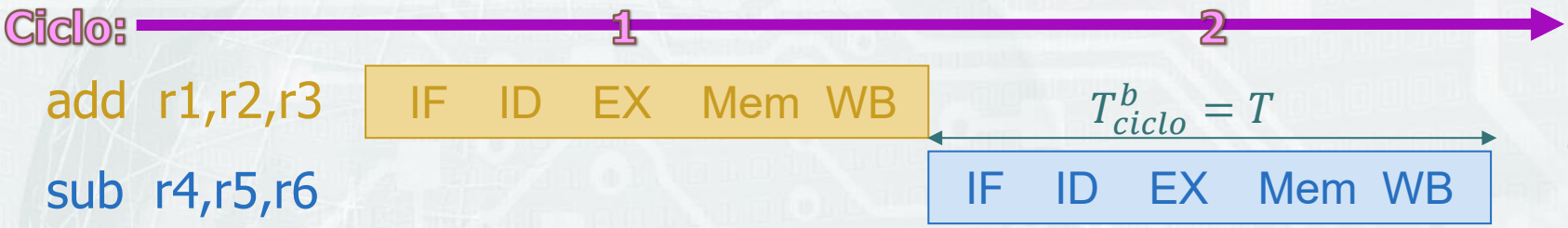
Núcleo **superescalar** con **2 emisiones por ciclo** y **5 etapas**:



$$S = \frac{T^b_{CPU}}{T^p_{CPU}} = \frac{NI \times 1 \times T}{NI \times \frac{1}{2} \times \frac{T}{5}} = 2 \times 5 = 10$$

$$S = emisiones_por_ciclo \times etapas_cauce$$

Mejora en un núcleo de procesamiento: unidades funcionales SIMD



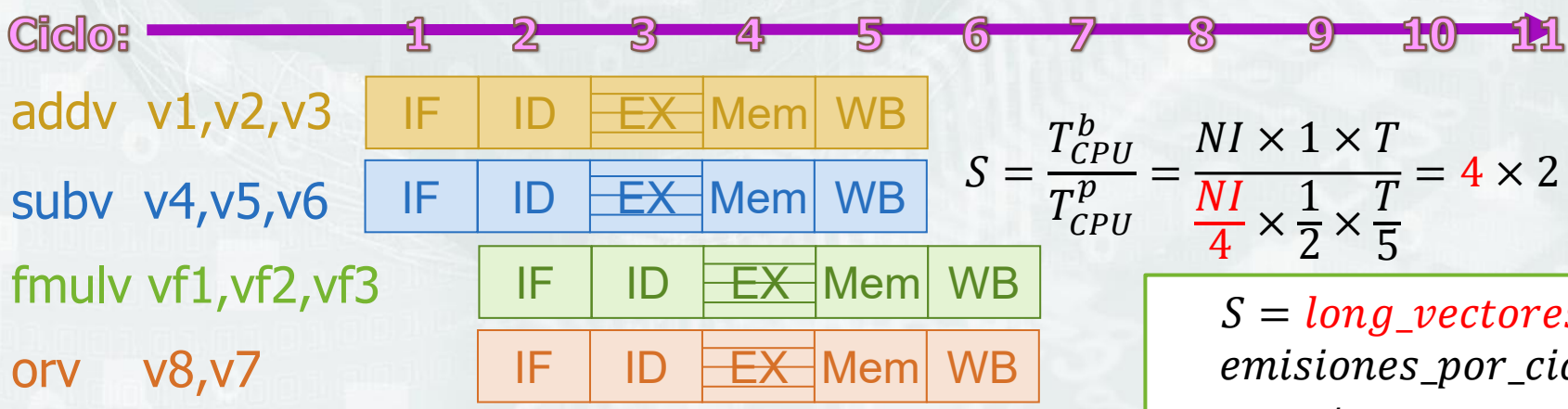
$NI^b = NI$

$CPI^b = 1$

$T^b_{ciclo} = T$

$$S = \frac{T^b_{CPU}}{T^p_{CPU}} = \frac{NI^b \times CPI^b \times T^b_{ciclo}}{NI^p \times CPI^p \times T^p_{ciclo}}$$

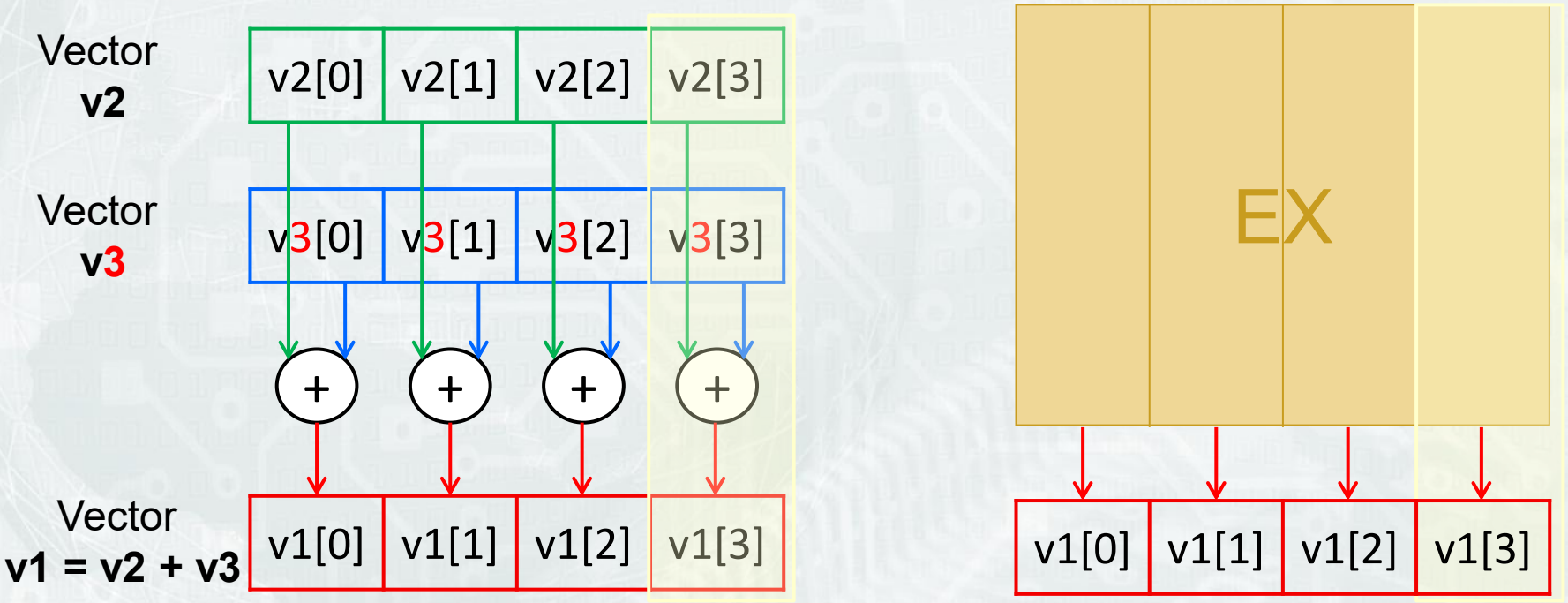
Núcleo **superescalar** con **2 emisiones por ciclo** y **5 etapas**, y **unidades funcionales SIMD** (vectoriales) que procesan **vectores de 4 componentes** (suponemos el mejor caso: el código genera sólo instrucciones que operan con vectores de 4 c.):



$$S = \frac{T^b_{CPU}}{T^p_{CPU}} = \frac{NI \times 1 \times T}{\frac{NI}{4} \times \frac{1}{2} \times \frac{T}{5}} = 4 \times 2 \times 5 = 40$$

$$S = long_vectores \times emisiones_por_ciclo \times etapas_cauce$$

Paralelismo de datos. Ej: suma de dos vectores

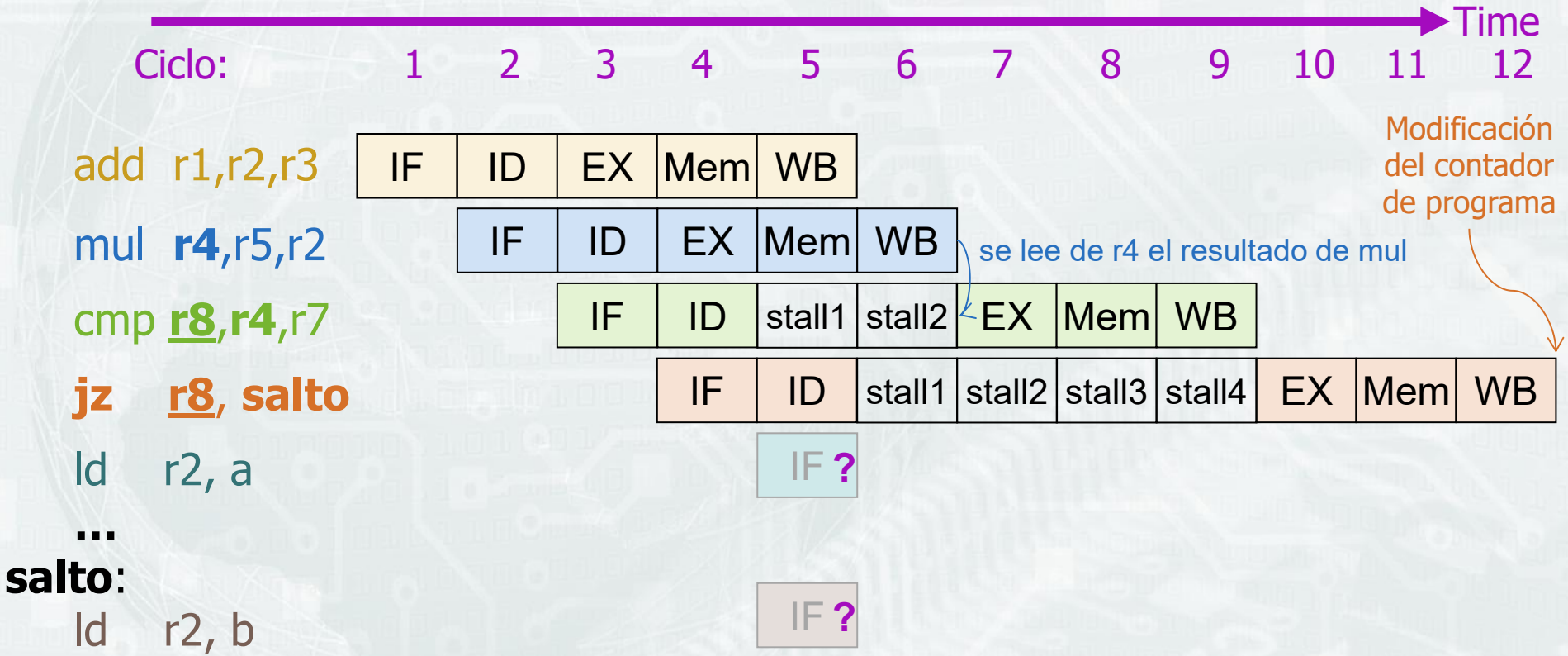


addv v1,v2,v3

¿Qué impide que se pueda obtener la ganancia en velocidad pico?

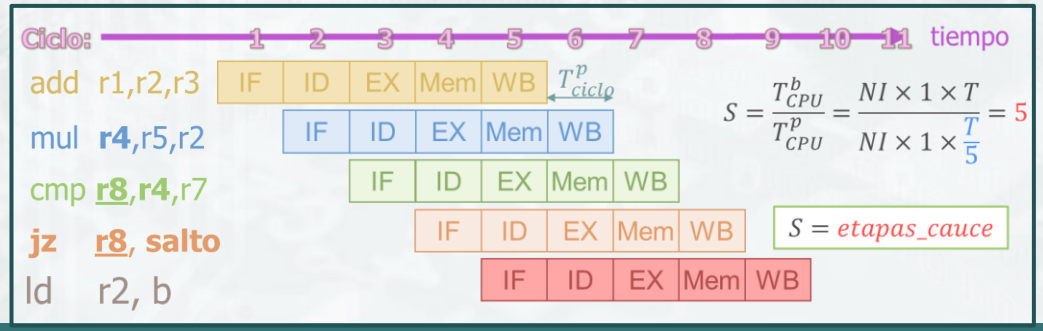
- Riesgos:
  - Datos
  - Control
  - Estructurales
- Accesos a memoria (debido a la jerarquía)

Riesgos (*hazards*): de datos, de control

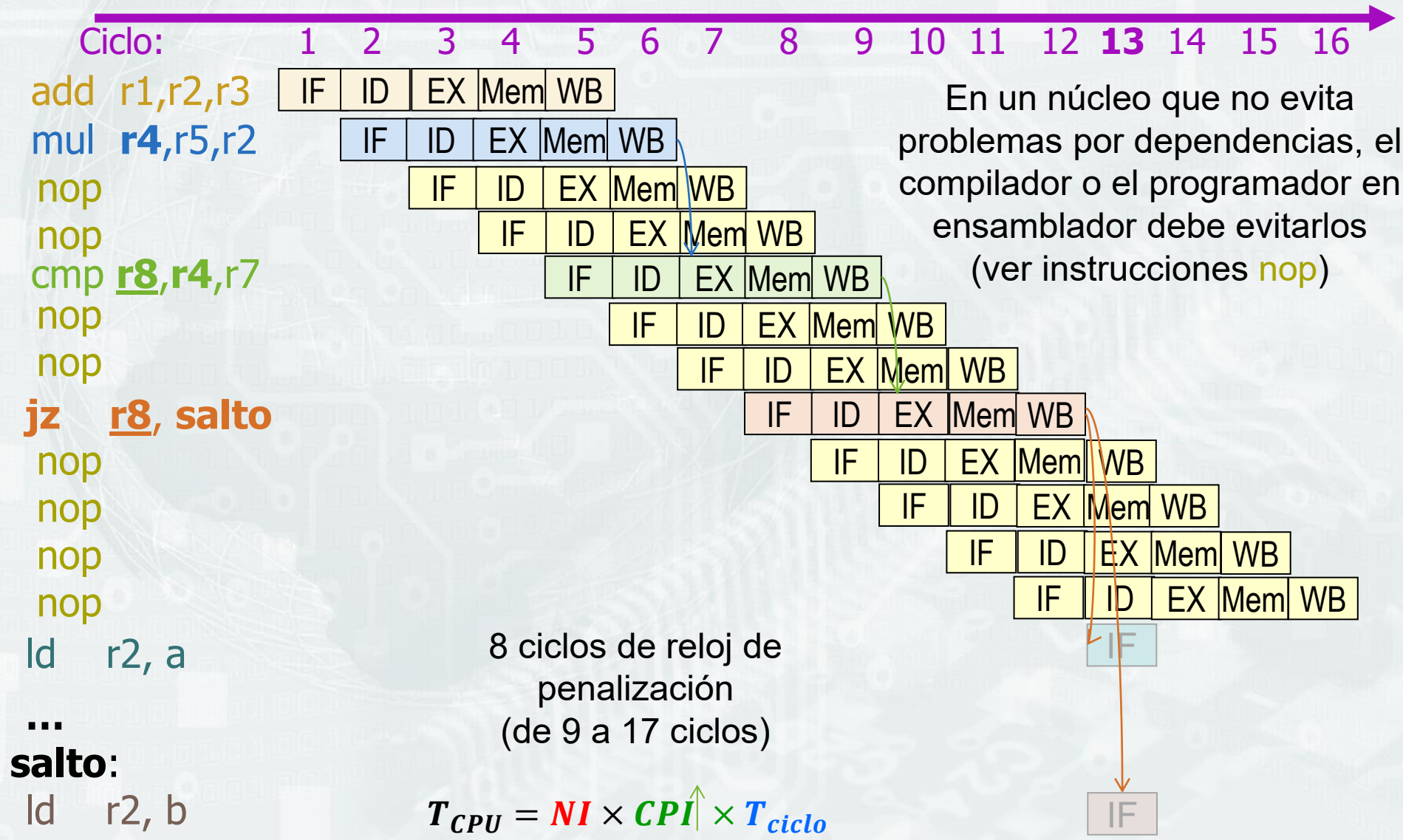


8 ciclos de reloj de penalización (de 9 a 17 ciclos)

$$T_{CPU} = NI \times CPI \times T_{ciclo}$$



Riesgos (*hazards*): de datos, de control

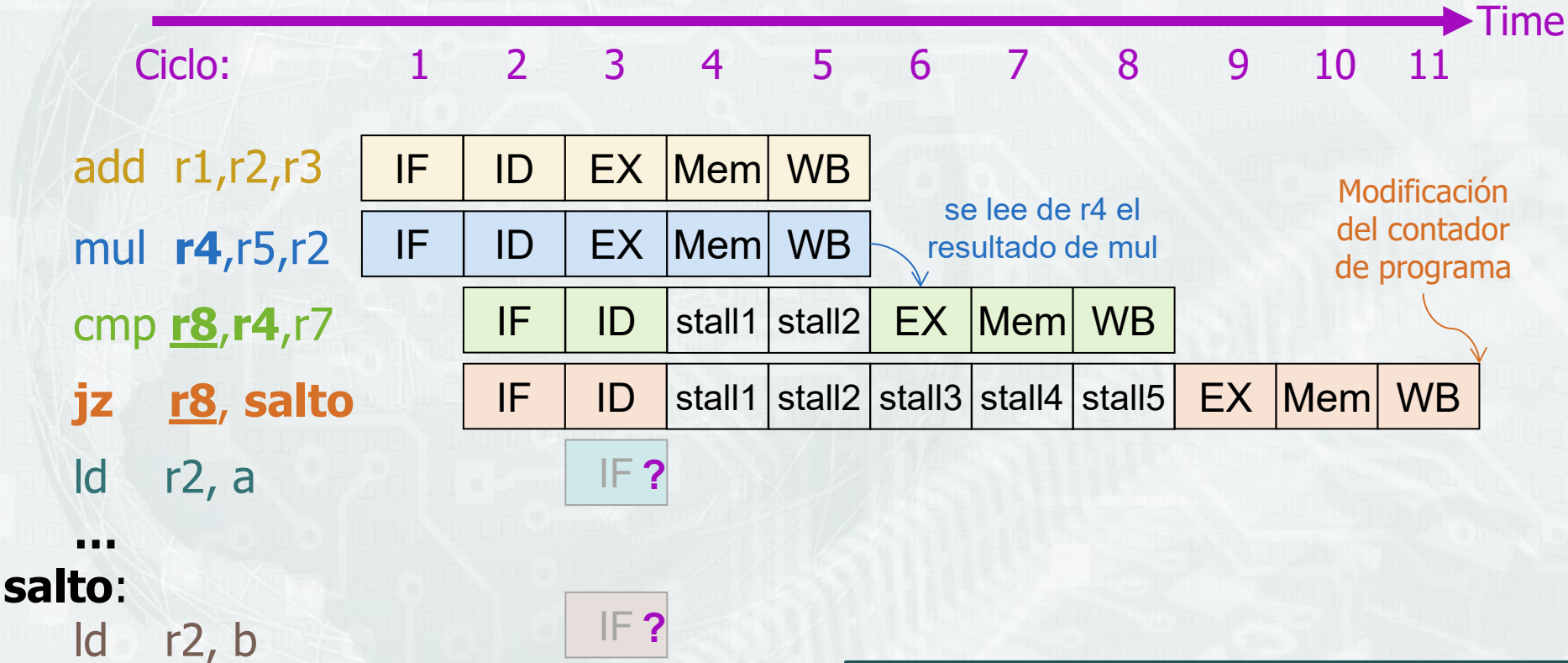


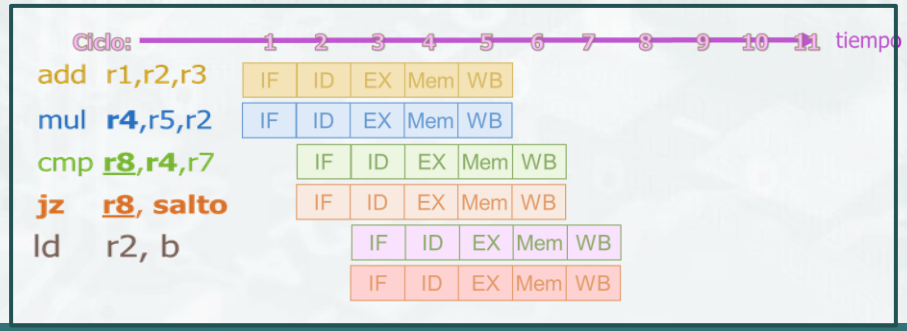
En un núcleo que no evita problemas por dependencias, el compilador o el programador en ensamblador debe evitarlos (ver instrucciones nop)

8 ciclos de reloj de penalización (de 9 a 17 ciclos)

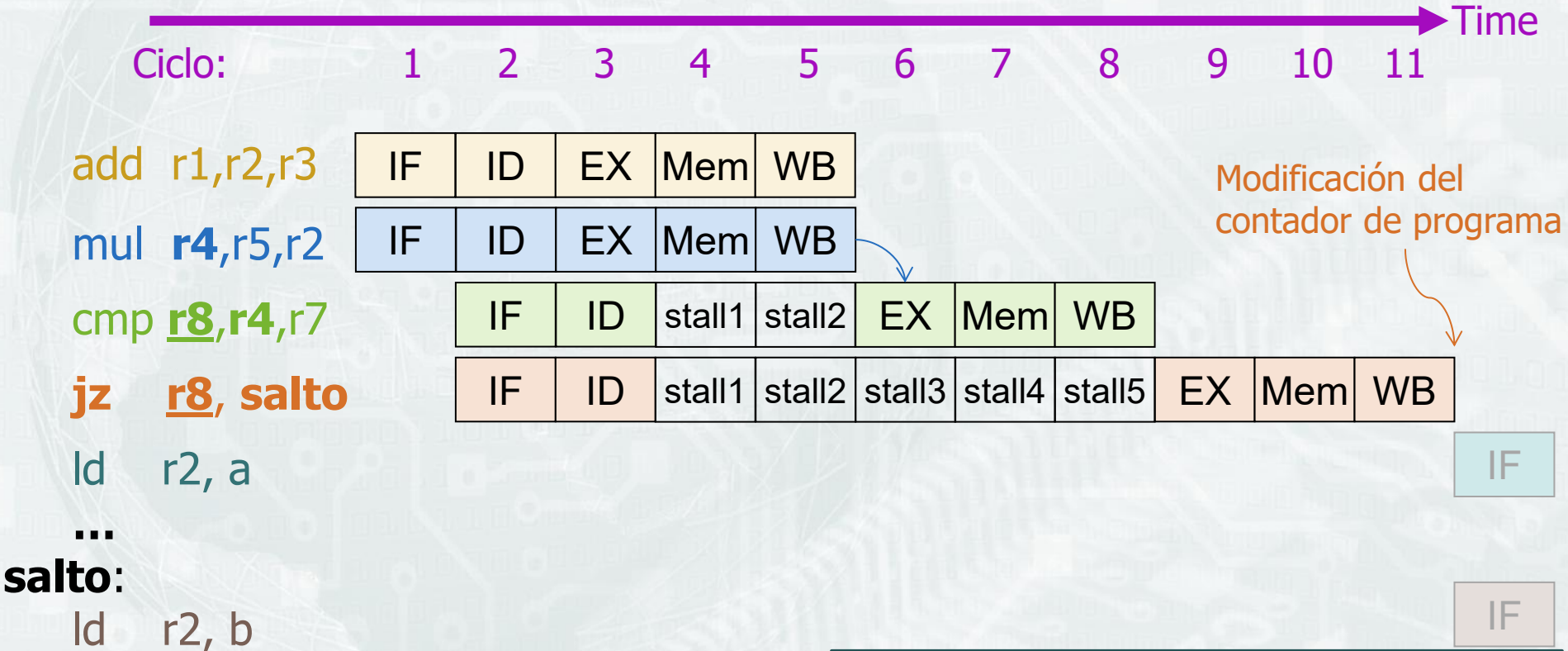
$$T_{CPU} = NI \times CPI \uparrow \times T_{ciclo}$$

Riesgos (*hazards*): de **datos**, de **control**

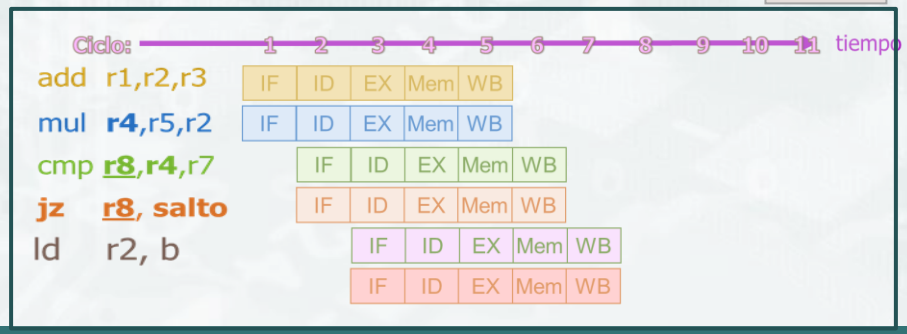




Riesgos (*hazards*): de **datos**, de **control**



9 ciclos de reloj de penalización (de 7 a 16)



Riesgos (*hazards*): de datos, de control, **estructural**

Ciclo:	1	2	3	4	5	6	7	8	9	10	11	12
add r1,r2,r3	IF	ID	EX	Mem	WB							
mul **r4**,r5,r2	IF	ID	stall1	EX	Mem	WB						
cmp **r8**,**r4**,r7		IF	ID	stall1	stall2	stall3	EX	Mem	WB			
jz **r8**, **salto**		IF	ID	stall1	stall2	stall3	stall4	stall5	stall6	EX	Mem	WB
ld r2, a			IF ?									
...												
salto:												
ld r2, b			IF ?									

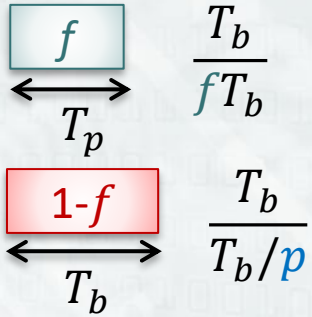
add y mul usan la misma unidad funcional

10 ciclos de reloj de penalización (de 7 a 17 ciclos)

Ley de Amdahl

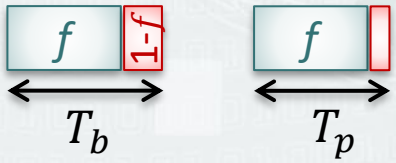
La mejora de velocidad, ***S***, que se puede obtener cuando se mejora un recurso de una máquina en un factor ***p*** está limitada por:

$$S = \frac{V_p}{V_b} = \frac{T_b}{T_p} \leq \frac{1}{f + \frac{1-f}{p}} = \frac{p}{1 + f(p-1)} \quad \xrightarrow[p\to\infty]{} \frac{1}{f} \quad \xrightarrow[f\to 0]{} p$$



donde ***f*** es la fracción del tiempo de ejecución del sistema base (i.e. de la máquina sin la mejora) durante el que no se usa dicha mejora.

Ejemplo: Si un programa pasa un 25% de su tiempo de ejecución en una máquina realizando instrucciones de coma flotante, y se mejora la máquina haciendo que estas instrucciones se ejecuten en la mitad de tiempo, entonces ***p*=2**, ***f*=0.75** y



$$\frac{(1-f)T_b}{p}$$

$$S = \frac{T_b}{T_p} = \frac{1}{0.75 + \frac{0.25}{2}} \approx 1.14$$

Habría que mejorar el caso más frecuente (lo que más se usa)

Contenido

- Lección 1. Clasificación del paralelismo implícito en una aplicación
- Lección 2. Clasificación de arquitecturas paralelas
- Lección 3. Evaluación de prestaciones y eficiencia de una arquitectura
  - Medidas usuales para evaluar prestaciones
  - Ganancia en prestaciones al realizar una mejora
  - **Eficiencia (coste, potencia, energía)**
  - Conjunto de programas de prueba (*Benchmark*)

Energía, potencia

- Un diseñador de procesadores (o chips) debe tener en cuenta lo siguiente al enfrentarse al diseño de un procesador:
  - Cuál es la potencia **máxima** que puede necesitar el procesador durante su funcionamiento (Potencia máxima real, *peak power*, *boost power*)
    - Afecta al sistema de suministro de potencia que necesita.
    - Afecta a las prestaciones máximas, prestaciones que alcanza cuando se acelera y, por tanto, afecta a las prestaciones cuando la carga de trabajo se presenta a ráfagas o hay más demanda en ciertos momentos (p. ej. al cargar la aplicación a ejecutar).
    - Denominaciones: Intel PL2 (*Power Level* 2) o MTP (*Maximum Turbo Power*), AMD PPT (*Package Power Tracking*), AMD GPU TBP (*Total Board Power*), Nvidia GPU TGP (*Total Graphics Power*).
  - Cuál es el consumo de potencia sostenido que podrá soportar (TDP: *Thermal Design Power*)
    - El sistema de refrigeración se dimensiona teniendo en cuenta este valor.
      - La energía eléctrica consumida por un procesador se trasforma en calor que debe ser disipado.
    - No es la potencia máxima real, que suele ser 1.5 veces mayor, ni la potencia media consumida durante una determinada tarea.
    - Afecta a la velocidad de procesamiento (prestaciones) de cargas de trabajo exigentes constantes (son cargas que no aparecen durante unos segundos, son constantes)
    - Denominaciones: Intel PL1 (*Power Level* 1) o PBP (*Processor Base Power*); AMD: PPT (*Package Power Tracking*)
  - Para obtener cuál procesador de dos es más eficiente energéticamente para una tarea (carga de trabajo) se debería tener en cuenta las prestaciones y el consumo de energía para esa tarea (Energía – julios o Wh, no potencia - vatios)
    - Eficiencia = Prestaciones / Energía (para una tarea) = (1/t) / Energía (= 1/EDP; EDP = *Energy Delay Product*)

$$\boldsymbol{Energía_operación} = \boldsymbol{NC} \times \boldsymbol{PPC} \times \boldsymbol{T}$$

$$\boldsymbol{Energía_operación} = \boldsymbol{Potencia} \times \boldsymbol{T}$$

NC: *Number of Computer*
PPC: *Average Power per Computer*
T :Tiempo de ejecución

Potencia, Energía, calor

Término	Qué representa	Unidad	Cuándo se usa	Relación con calor (Q) generado	Otras denominación y variantes	Ejemplo (IntelArrow)
Frecuencia (f)	Velocidad emisión instrucciones, velocidad conmutación transistores.	Hz (c/s)	Frecuencia turbo: durante intervalos de tiempo cortos. Frecuencia base/sostenida: para mantener la potencia sostenida	↑f => ↑Q (lineal)	*Boost clock/boost frequency/turbo frequency; base clock/base frequency*	Turbo: 5.1 GHz. Base: 2 GHz
Potencia sostenida (Psostenida)	Límite de potencia para periodos largos de tiempo, para carga sostenida (normal)	W (J/s)	Carga de trabajo normal y sostenida, prestaciones sostenidas.	↑P => ↑Q (lineal, E = P•t)	Intel PL1; AMD PPT (usa también TDC, *Thermal Design Current*) . Pbase	28W
TDP *Thermal Design Power*	Guía para el diseño del SR, VRM (o PMIC si SoC).	W	Diseño del SR, VRM. También se usa como Psostenida	Calor a disipar por SR	Relacionado con PL1 y PPT	28W
Potencia máxima (Pmáx)	Límite de potencia para periodos cortos (potencia turbo)	W	Intervalos cortos de tiempo (ej. 28 s) para acelerar ejecución, prestaciones pico (ej. lanza app, pico en juegos, ráfagas de computación cortas,…)	↑P => ↑Q	Intel PL2; AMD usa corriente EDC (*Electrical Design Current*)	**115W**
Potencia media	Media potencias alcanzadas en un periodo	W	Determinar repercusión en consumo de luz, duración batería.	↑P => ↑Q		
Tj.max *Maximum Junction Temperature*	Temperatura máxima permitida en el silicio (en las uniones dentro de transistores)	ºC	Cuando Tj se aproxima o alcanza Tj.max, DVFS ↓ f/V (potencia)	↑Q => ↑ Tj		105ºC
Energía	Potencia consumida para una tarea o tareas habituales	J	Para comparar dos procesadores. Depende del tiempo en Pmáx y Psostenida	Q ≈ E en circuitos electrónicos		28s x **115W** + 72s x 28W =92176 J
Corriente máxima	Intel IccMax: Corriente instantánea máxima permitida en CPU. AMD EDC: Corriente máxima para turbo	A	Si I se aprox. a IccMax, VRM ↓V (peligra CPU/VRM). Si I supera EDC, DVFS manda ↓ f/V (peligra VRM, no CPU)	↑I^2 => ↑Q (cuadrático) (E = P•t = V•I•t = I^2•R•t)	Intel IccMax (*Max. Core Current*), AMD EDC (*Electrical Design Current*)	64 A

Potencia dinámica. Gestión calor generado, ajuste de frecuencia/voltaje (potencia)

Nodo (lógico) en este contexto es una conexión entre varios componentes del circuito electrónico. Su C incluye la capacitancia de puertas, drenadores, fuentes y líneas de conexión.

$C_{nodo} = C_{gate} + C_{diff} + C_{wire}$ C_{wire}= C línea conexión

C_{diff}= C entre drenador y substrato, y fuente y substrato

Energía consumida en una transición (1->0->1 o 0->1->0) de un nodo:

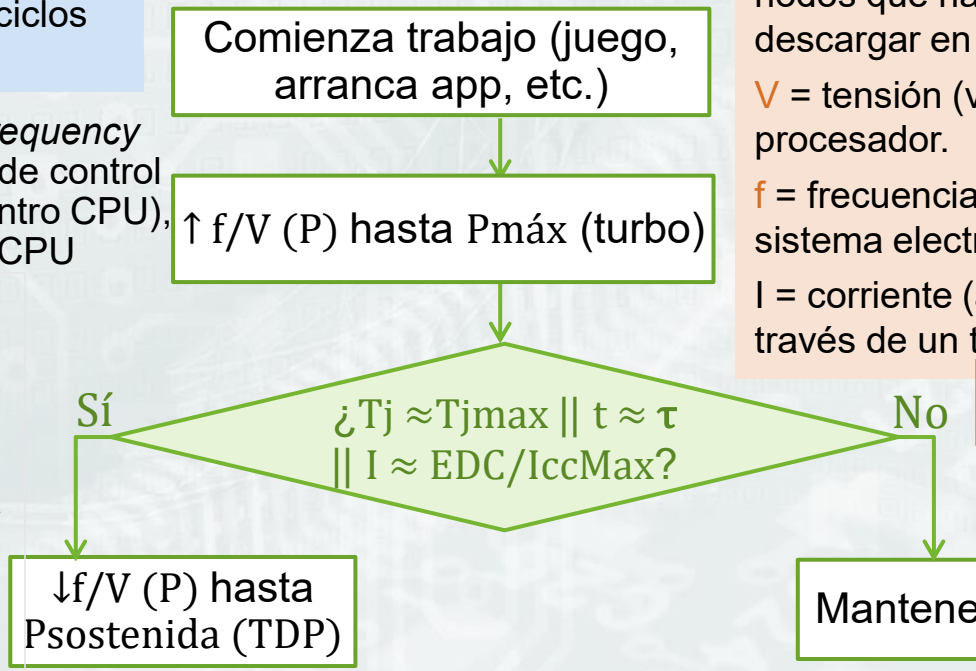
$$E_{transición_nodo} = C \bullet V^2 \quad (P = V \bullet I = V \bullet Q/t = C \bullet V^2/t)$$

P_{nodo}: Potencia dinámica para un nodo. Si conmutara todos los ciclos (α=1): $P_{nodo}(\alpha=1) = C \bullet V^2 \bullet F$

DVFS (*Dynamic Voltage and Frequency Scaling*): microcódigo + unidad de control que lo ejecuta (ambos están dentro CPU), decide y ordena: V a VRM; F a CPU

VRM (*Voltage Regulator Module*): hardware en la placa, DVFS le dice V a generar (ejecuta órdenes de DVFS), pero VRM avisa si I se aprox. al máximo o si se sobrecalienta y, entonces, DVFS ajusta V y F.

PMIC (*Power Management Integrated Circuits*): versión en el chip con circuito a controlar



Esquema de gestión simplificado, importantes variaciones entre fabricantes de procesadores

P = J/s (W) de energía generados y calor disipado por un sistema electrónico (dinámica, sin estática).

N = nº de nodos en el sistema electrónico.

α = coeficiente de actividad, probabilidad de conmutación de un nodo en un ciclo.

C = capacitancia (faradios) de los nodos que hay que cargar y descargar en una transición

V = tensión (voltios) suministrada al procesador.

f = frecuencia (hercios) de reloj del sistema electrónico.

I = corriente (amperios) que fluye a través de un transistor en transición

$$P \approx N \bullet \alpha \bullet C \bullet V^2 \bullet f$$

Posibles valores para laptop

Término	Qué representa	Uni-dad	Cuándo se usa	Ejemplos
Voltaje	El voltaje generado por VRM (o PMIC) para la electrónica a partir del voltaje de la fuente de alimentación.	V	Se usa para calcular la potencia consumida por un sistema electrónico (CPU, memoria, etc.)	Intel Arrow Lake: Psostenida 0.65-0.9V; Pmáx 1.1-1.25V
C	Capacidad eléctrica o capacitancia. $C_{nodo} = C_{gate} + C_{diff} + C_{wire}$	F, fF	Se usa para calcular la potencia consumida por un sistema electrónico	Nodo pequeño (7nm, 5nm): 1-5fF; bus local corto: 5-20fF; Bus largo:50-200fF; reloj: cientos fF
α (coeficiente actividad)	Probabilidad de que un nodo conmute en un ciclo de reloj		Se usa para aproximar la potencia consumida por un sistema electrónico (CPU, memoria, etc.)	Señal de reloj: α=1. Promedio en circuitos reales (CPU, SoC): α entre 0.05 y 0.15
τ (tiempo permitido en turbo)	Tiempo máximo permitido con Pmáx. (Intel)	s	Cuando el tiempo en Pmáx alcanza τ, DVFS baja f/v (potencia) hasta Psostenida (Intel)	28s (Intel Arrow)
Nodo lógico	Conexión entre componentes del circuito eléctrico (tiene voltaje único). También llamado Net o netlist (lista de conexiones)		Se usa para aproximar la potencia consumida por un sistema electrónico (CPU, memoria, etc.)	Nodo pequeño: p. ej- nodo entre dos inversores o entre dos nand de 2 entradas, etc.
Nº de transistores	Los circuitos electrónicos se componen de transistores		Se usa para calcular una cota superior de la potencia consumida.	Apple M2 (5nm) = 20×10^9 AMD Ryzen 7000 Series (5nm) = 6.5×10^9 ($13.1.5 \times 10^9$ para escritorio)
Nº de nodos pequeños	Nodos sencillos formados por conexiones entre inversores, nand, etc.		Se usa para aproximar la potencia consumida por un sistema electrónico (CPU, memoria, etc.).	Nodo pequeño: entre 0.3 y 0.6 del nº de transistores del sistema.

Ver Wikipedia, Intel Core Ultra 7 255H Specs | TechPowerUp CPU Database

Coste de producción/fabricación, precio

- El **coste** de fabricación o producción de un dispositivo (ya sea de un chip, tarjeta, placa, o sistema) decrece desde que empieza su venta (empieza a utilizarse):
  - El coste decrece por la "curva de aprendizaje" (la producción de dispositivos va detectando y eliminando errores de diseño y de fabricación).
    - Una forma de medir la "curva de aprendizaje" es calculando el porcentaje de dispositivos fabricados que superan el test que comprueba tras la fabricación que el dispositivo funciona correctamente.
  - En el coste influye:
    - Coste de ingeniería no recurrente.
      - Coste que se produce desde la idea de un producto hasta la fase previa a su lanzamiento = Coste debido a la investigación, diseño, desarrollo y test de un nuevo dispositivo antes de comenzar su producción masiva (NRE cost = *Non-Recurring Engineering cost*).
    - Coste de producción por unidad.
  - Conforme aumenta el volumen de fabricación se van depurando errores y, con ello, se va amortizando el coste NRE y se va reduciendo el coste de producción.
- El **precio** depende del coste y de otros parámetros, como la demanda o la competencia.

Lecciones

- Lección 1. Clasificación del paralelismo implícito en una aplicación
- Lección 2. Clasificación de arquitecturas paralelas
- Lección 3. Evaluación de prestaciones de una arquitectura
  - Medidas usuales para evaluar prestaciones
  - Ganancia en prestaciones al realizar una mejora
  - Eficiencia (coste, potencia, energía)
  - **Conjunto de programas de prueba (*Benchmark*)**

Benchmarks

- Propiedades exigidas a medidas de prestaciones:
  - Fiabilidad => *Representativas, evaluar diferentes componentes del sistema* y *reproducibles*
  - Permitir comparar diferentes realizaciones de un sistema o diferentes sistemas => Aceptadas por todos los interesados (usuarios, fabricantes, vendedores)
- Interesados:
  - Vendedores y fabricantes de hardware o software.
  - Investigadores de hardware o software.
  - Compradores de hardware o software.

Tipos de *Benchmarks*

- Tipos de Benchmark:
  - De bajo nivel o microbenchmark
    - test ping-pong, evaluación de las operaciones con enteros o con flotantes
  - Kernels
    - resolución de sistemas de ecuaciones, multiplicación de matrices, FFT, descomposición LU
  - Sintéticos
    - Dhrystone, Whetstone
  - Programas reales
    - Ej. SPEC CPU2017 (https://www.spec.org/): enteros (gcc, perlbmk),
  - Aplicaciones diseñadas
    - Predicción de tiempo, dinámica de fluidos, animación etc. (p. ej. SPEC2017).